Resource-Versioned Outcome-Aware Metacontrol for Software Action Systems

Felipe Santibáñez-Leal
CAOS open-research programme

Preprint · v0.02, 30 October 2026 · Licence CC BY 4.0
CAOS open-research programme · Neuraxis, software-action metacontrol
Version DOI `10.5281/zenodo.23821087` · concept DOI (always latest)
`10.5281/zenodo.23784117`
Not peer reviewed. Source code, data and computational records:
`github.com/fsantibanezleal/CAOS_Neuraxis`

Abstract

Software agents must choose admissible actions quickly while deciding whether uncertainty, novelty, or anticipated consequences justify slower computation. We specify a resource-versioned, outcome-aware metacontroller (M12) that combines recurrent action preference, bounded model-based lookahead, outcome-sensitive delegation, explicit surprise detection, asynchronous planner proposals, and validation against current resource revisions. We give its decision contract, state transitions, allocation objective, and falsifiable evaluation protocol, then report the existing matched benchmark evidence. On 1,200 core episodes, M12 solved 1,160 while a guarded finite-state controller solved 1,200; median decision latencies were 18.13 ms and 0.0219 ms, respectively. Four component ablations matched M12 task success, and the fitted deferral gate selected never-defer. On 24 structural-transfer episodes, XGBoost solved 24 and M12 solved 21. Thus this implementation has not shown a task-success or efficiency advantage. The paper contributes a concrete method specification and a negative result that identifies where the allocation hypothesis remains untested.

Keywords: metacontrol, learning to defer, software agents, resource versioning, asynchronous planning, negative results

1 Problem formulation

Let a software environment at decision time t expose observation o_t, candidate set A_t, and a vector of resource revisions v_t. A fast controller selects an admissible action $a_t \in A_t$ from a typed action contract. A deliberative planner may return a proposal after the environment changes. The controller must both select actions and decide when additional computation is worth its cost. We distinguish (i) authority, enforced by the action broker, from (ii) policy quality, measured by environment outcomes. A valid proposal is not necessarily useful; a useful proposal is not admissible unless it satisfies the current contract.

The central question is whether outcome-aware, stateful allocation can improve the joint frontier of task completion, decision latency, deliberation cost, and effect correctness relative to fixed fast policies and fixed delegation. The hypothesis is conditional: for environments with consequential

uncertainty and variable tool behavior, a controller that predicts incremental value and invalidates stale proposals may outperform fixed policies under a matched budget. The current benchmark does not instantiate enough consequential uncertainty to establish that claim.

2 Relation to prior methods

Dual-controller accounts distinguish cached or habitual responses from model-based planning, while computational accounts study arbitration as a function of uncertainty and the relative reliability of model-free and model-based values [1, 2, 3]. These findings motivate functional analogies, not a claim that a software controller reproduces neural mechanisms. Predictive-processing and recurrent-control theories offer candidate descriptions for residual-driven adaptation and persistent state [4, 5]. Biological evidence does not determine the software architecture or validate its performance.

Machine-learning systems that defer to experts frame delegation as a selective prediction problem with a cost-sensitive decision boundary [6, 7]. Rational metareasoning treats computation itself as an action whose expected value must be weighed against its cost [8]. Hierarchical reinforcement learning supplies temporal abstractions for policies whose actions persist over multiple environment steps [9]. These approaches typically evaluate the deferral decision or learned computation policy; the proposed controller adds explicit resource-revision validity and observed planner outcomes to the action-level loop. Asynchronous agent architectures and tool-use benchmarks provide complementary engineering and evaluation precedents, but are not direct replications of the present benchmark [10, 11].

3 Resource-versioned outcome-aware metacontrol

3.1 Controller state and fast action score

The controller maintains bounded episode-local recurrent state h_t, a residual statistic r_t, the latest resource revision vector v_t, and at most one active planner request. A recurrent policy emits candidate probabilities $p_\theta(a \mid o_t, h_t)$. A learned transition model supports bounded lookahead utility $U_\phi(a \mid o_t)$ over the same admissible candidates. The implementation combines these signals as a ranking score, not as a calibrated probability:

$$S_t(a) = \log(p_\theta(a \mid o_t, h_t) + \epsilon) + \lambda U_\phi(a \mid o_t), \qquad a \in A_t. \tag{1}$$

The selected candidate is still passed through the native broker, which independently checks authority and current resource state.

3.2 Outcome-aware allocation

For a candidate computation q (a bounded planner request), define its incremental value relative to immediate execution as $\Delta_t(q) = \mathbb{E}[R \mid q, o_t, A_t] - \mathbb{E}[R \mid \text{fast}, o_t, A_t] - c(q)$, where R is a task-level reward and $c(q)$ includes latency and resource cost. A cost-sensitive gate is fitted to paired executed branches and separately calibrated and threshold-selected. The gate is permitted to abstain from delegation. The implementation also has an explicit surprise/evidence trigger based on normalized prediction residuals; this trigger is distinct from the learned deferral gate. In the evaluated fit, the learned threshold was 1.01, producing never-defer behavior. The existence of a separate trigger does not change that gate result.

3.3 Asynchronous proposals and validity

A planner request is bound to a goal identifier, observation revision, candidate-resource versions, request deadline, and expiration time. While a proposal is pending, affected work is held when required by the operation contract. On arrival, the controller checks goal identity, observation revision, candidate resources, and the executor's fresh resource versions. A mismatch discards the proposal and resumes fast control from the current observation. An urgent cancellation may proceed and invalidate the pending proposal. The implementation polls within a bounded per-call interval and caps request count and lifetime. The broker remains the final authority boundary for all methods.

3.4 Operational algorithm

1. Read the current observation, candidate set, and resource revisions; update recurrent state and prediction residual.
2. Score only currently admissible candidates using the recurrent preference and bounded lookahead utility.
3. If a proposal is pending, validate its bindings against current revisions. Use it only if every binding is current and its action remains admissible; otherwise invalidate it.
4. Evaluate whether an eligible deliberation request has positive calibrated incremental value or meets the separate surprise/evidence trigger. Respect budget, deadline, and request limits.
5. Submit the selected fast or validated planner action to the broker. Record request, proposal, invalidation, admission, effects, postconditions, latency, and reward.
6. Update the outcome model only from executed, identity-aligned outcomes; do not treat an unexecuted proposal as task reward.

4 Evaluation protocol

The frozen core contains 20 task families, six variants, and ten seeds for each of twelve methods (14,400 episodes). The evaluation includes guarded finite-state and behavior-tree controllers; logistic, gradient-boosted, recurrent, bandit, PPO, and model-based policies; a language-model planner; fixed asynchronous delegation; a learned deferral gate; and M12. All methods share the native broker and task contracts. Structural-transfer episodes and ablations are reported separately, not pooled with the core. Primary outcomes are independently checked task postconditions, decision latency, planner use, cost, cancellation, and effect correctness. The data, split boundaries, executable pipeline, and hashes are available in the repository and the companion technical report.

5 Results

M12 solved 1,160 of 1,200 core episodes (96.67

6 Interpretation

The observed outcomes refute the benchmark-specific prediction that M12 improves task completion or decision efficiency over the competent fixed baseline. The failure concentration in long-job

Table 1: Selected outcomes from the frozen evaluation.

Method	Core success	p50 decision (ms)	Transfer success
M01 guarded finite-state	1200/1200	0.0219	not pooled
M04 gradient-boosted policy	1200/1200	see result artifact	24/24
M12	1160/1200	18.1303	21/24

cancellation points to a concrete interaction between pending work, cancellation acknowledgement, and resource revision. Identical task success across ablations means the current results do not identify a task-success contribution from the proposed components. Transfer is small and favors M04. These findings do not show that outcome-aware allocation is ineffective in every environment; they show that the present protocol and workload do not establish its value.

7 Limitations and discriminating experiments

The core tasks are saturated by M01, making success improvements unidentifiable on most cases. Ten seeds yield identical outcomes in many method-case-variant cells, so episode count overstates independent variation. Only seven positive labels informed the gate. The transfer sample is small. The current evidence therefore cannot support a broad generalization claim.

A discriminating evaluation should create held-out task families with ambiguous specifications, changing tool behavior, conflicting resource revisions, variable planner quality, and costly but recoverable failures. It should include both cases where planning changes the verified outcome and cases where delegation is wasteful or harmful. Train, calibrate, select thresholds, and evaluate on disjoint families. Randomize meaningful transition and tool outcomes across seeds. Match compute budgets; report success, latency, total cost, effect correctness, recovery, invalidation, and planner utilization. Pre-register a minimum effect and uncertainty procedure at the family level. Stress-test stale proposals and cancellation under injected delays. A positive result requires a paired advantage over M01 and M04 without degraded authority or postcondition guarantees.

8 Conclusion

M12 specifies a software metacontroller that couples fast recurrent action selection with bounded lookahead, outcome-sensitive allocation, asynchronous delegation, and explicit resource-version validation. Its implementation has not demonstrated an advantage on the current benchmark: a guarded finite-state controller is faster and solves every core task, while the proposed controller fails a cancellation subset and its ablations do not change task success. The contribution is a falsifiable method design and a set of benchmark-derived constraints for a more discriminating evaluation. No conclusion about general intelligence or biological equivalence follows.

Data and code availability

Source code, frozen traces, derived indexes, evaluation scripts, and runtime contracts are available at `https://github.com/fsantibanezleal/CAOS_Neuraxis`. The separately versioned engine is available at `https://github.com/fsantibanezleal/reflexmesh`. The product-wide methods, protocol, and benchmark analysis are available in the detailed Neuraxis technical report.

References

[1] N. D. Daw, Y. Niv, and P. Dayan. Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control. *Nature Neuroscience*, 8:1704–1711, 2005. doi:10.1038/nn1560.

[2] S. W. Lee, S. Shimojo, and J. P. O'Doherty. Neural computations underlying arbitration between model-based and model-free learning. *Neuron*, 81:687–699, 2014. doi:10.1016/j.neuron.2013.11.028.

[3] W. Kool, F. A. Cushman, and S. J. Gershman. When does model-based control pay off? *PLoS Computational Biology*, 12:e1005090, 2016. doi:10.1371/journal.pcbi.1005090.

[4] R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex. *Nature Neuroscience*, 2:79–87, 1999. doi:10.1038/4580.

[5] V. Mante et al. Context-dependent computation by recurrent dynamics in prefrontal cortex. *Nature*, 503:78–84, 2013. doi:10.1038/nature12742.

[6] D. Madras, T. Pitassi, and R. Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *NeurIPS*, 2018.

[7] H. Mozannar and D. Sontag. Consistent estimators for learning to defer to an expert. In *ICML*, 2020. URL: `https://proceedings.mlr.press/v119/mozannar20b.html`.

[8] S. Russell and E. Wefald. Principles of metareasoning. *Artificial Intelligence*, 49:361–395, 1991.

[9] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112:181–211, 1999. doi:10.1016/S0004-3702(99)00052-1.

[10] S. Zhang et al. Leveraging dual process theory in language agent framework for real-time simultaneous human-AI collaboration. In *ACL*, 2025. URL: `https://aclanthology.org/2025.acl-long.206/`.

[11] Y. Lu et al. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv:2408.04682, 2024.